Università degli Studi di Trento Corso di Programmazione I – E3: 145430/145935 1° Appello – 01/01/2020 – a.a. 2019/2020	**PC N°**
COGNOME ..... **NOME .....** **MATRICOLA** ☐☐☐☐☐☐	
RITIRATO ☐ **FIRMA PER RITIRO.....**	

Completare le voci **Cognome**, **Nome** e **Matricola**, se tali voci saranno mancanti o non-leggibili elaborato **NON** verrà corretto.
NON è consentito utilizzo di alcun strumento cartaceo, elettronico, informatico etc. durante la prova.
Fa parte della valutazione la leggibilità del codice C/C++.
Scrivere le risposte su questo foglio e riconsegnare. I risultati verranno pubblicati su DOL
Tempo a disposizione: 120 minuti.

PARTE TEORICA: 13 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Domanda 1 – punti 2] Calcolare la somma tra $(75)_8$ e $(22)_8$.
Inoltre dire a quali delle soluzioni equivale il risultato, mostrando tutti i passaggi per la somma e le conversioni

SI ☐ NO ☐ $(117)_8$
SI ☐ NO ☐ $(100111)_2$
SI ☐ NO ☐ $(6F)_{16}$
SI ☐ NO ☐ $(79)_{10}$
!

[Domanda 2 - punti 4] Si scriva esattamente l'output (istruzioni ***printf***), nel riquadro a fianco, che viene prodotto durante l'esecuzione del programma se venisse invocata la funzione **MergeSort(A, 0,3**), con A un array di interi contenente le cifre della propria matricola (inserirlo negli appositi spazi).
NOTA: si suppone che la funzione Merge sia implementata (ricordarne il funzionamento).

	0	1	2	3	4	5
A						

```
void MergeSort (int A[], int p, int r){
  if (p < r) {
   int q = (p+r)/2);
   printf("(p=%d, r=%d, q=%d)\n",p,r,q);
   MergeSort (A, p, q);
   MergeSort (A, q+1, r);
   Merge (A, p, q, r);
  }
}
```

[Domanda 3 – punti 3] Risolvere la seguente equazione di ricorrenza, specificando il limite asintotico superiore.

$$\begin{cases} T(1) = 1 \\ T(N) = 2*T(N/2)+2 \end{cases}$$

[Domanda 4 - punti 4]
Data la seguente struttura:

```
typedef struct Tnodo{
  int dato;
  Tnodo *DX, *SX;
} Tnodo;
```

[A] Scrivere una funzione ricorsiva che analizza un generico albero e dimezza il valore "`dato`" presente nel nodo se questo è pari:

[B] Data la sequenza di valori:

`4, 3, 3, 4, 2, 6, 6, 2 ,5`

costruire un albero binario di ricerca (BST), inserendo nell'albero nell'esatto ordine i valori riportati nella sequenza, mettendo i valori uguali nel sottoramo di DX.

[C] Per l'albero così realizzato inoltre calcolare

Cammino									
Altezza									
Visita pre-ordine									
Visita post-ordine									
Visita in-ordine									
Visita level-ordine									

PARTE PRATICA: 20 PUNTI – PUNTEGGIO MINIMO 10 PUNTI

[A - punti 3] *Inserire i seguenti tipi in un file* ***dati.h*** **e implementare i metodi di stampa ed i costruttori:**

```
typedef enum Tsistoper{ WIN, MAC, LINUX } Tsistoper;
typedef struct Tserver{
  char hostname[20];
  int numCPU, ram;
  Tsistoper so;
  Tserver ()  { /*  da implementare */ }
  void stampa() { /*  da implementare → formato: vedi punto E */ }
}Tserver;
typedef struct Tcloud{  //elemento base di lista
  Tserver server;
  Tcloud *next;
  Tcloud () { /*  da implementare */ }
  Tcloud (Tserver s, Tcloud* n) { /*  da implementare */ }
  void stampa() { /*  da implementare */ }
} Tcloud;
```

[B - punti 1] *Modificare il file* ***main.cpp*** *includendo il file* ***dati.h*** *ed inserendo il seguente codice nel* ***main****:*

```
Tcloud* lst = NULL;
Tserver s;
for(int i=0; i<10; i++) { creaServer(&s); lst=inserisciServer(lst, s); }
stampaServer(lst);
salvaServer(lst);
serverEquivalente(lst, MAC).stampa();
```

[C - punti 3] Implementare una funzione **`creaServer`** che riceve in input per riferimento una variabile di tipo **Tserver s** e la inizializza con:

- campo **so** valore casuale tra WIN, MAC, LINUX;
- campo **numCPU** valore casuale tra 4 e 8 (inclusi).
- campo **ram** valore letto da tastiera, tra 4 e 16 (inclusi) e pari, con controllo input;
- campo **hostname** letto da tastiera.

[D - punti 3] Implementare una funzione **`inserisciServer`** che riceve come input una variabile di tipo **`Tcloud*` pc** (lista) e variabile di tipo **`Tserver` s** ed inserisce il server **s** nella lista **pc** (*insertLast*). La funzione restituisce l'indirizzo della lista aggiornata.

[E - punti 2] Implementare una funzione **`stampaServer`** che riceve in input una variabile di tipo **`Tcloud`*** **pc** (lista) e stampa il contenuto della lista utilizzando opportunamente il metodo **stampa** di **`Tcloud`**. Nella forma (per l'enum stampare etichetta in modo opportuno):

`hostname (so)` **RAM=**`ram` **CPU=**`cpu`

[F - punti 4] Implementare una funzione **`salvaServer`** che riceve in input una variabile di tipo **`Tcloud*` pc** (lista) e salva su file tutti i server che hanno il valore di **ram** > 8 e **numCPU** = 6. Inserire i dati in un file denominato "**server.txt**", nella forma

`hostname (so)` **RAM=**`ram` **CPU=**`cpu`

[G - punti 4] Implementare una funzione **`serverEquivalente`** che riceve in input una variabile di tipo **`Tcloud*` pc** (lista) ed un valore di tipo **`Tsistoper` sis**. Restituisce un server con **so** pari al parametro **sis**; **ram** e **numCPU** pari alla somma rispettivamente di RAM e CPU dei server che hanno campo so uguale al parametro sis ed **hostname** pari a "stringa vuota".

È possibile creare strutture, funzioni, metodi aggiuntivi per realizzare gli esercizi proposti.
Il codice commentato NON verrà corretto!